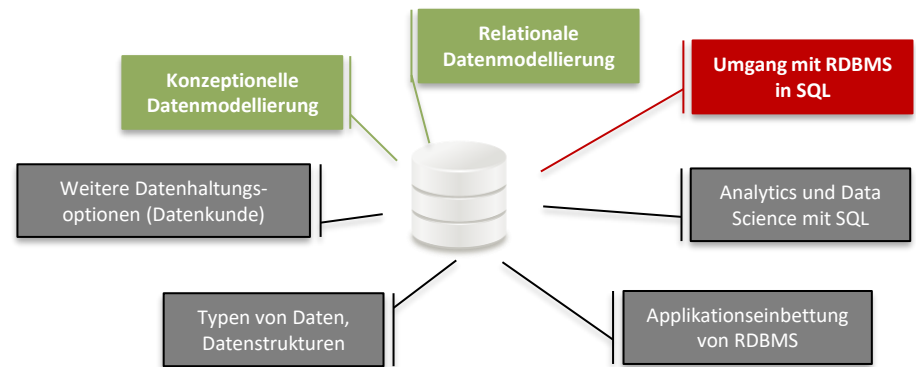


DSCB140
Datenbanken und Datenkunde I

VL4 – Structured Query Language (SQL)
Teil 1: RDBMS-/SQL-Komponenten & DDL

Umgang mit RDBMS in SQL Teil 1: DDL & DQL

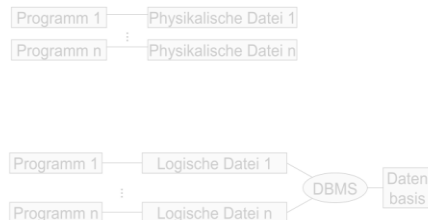


- Den Umgang mit Datenbankmanagementsystemen am Beispiel ausgewählter Lösungen erlernen.
- Datenbankschemata in SQL anhand der Data Definition Language (DDL) definieren können.
- Datenbanken anhand der Data Manipulation Language (DML) mit Inhalten befüllen sowie Inhalte in Datenbanken manipulieren (löschen, ändern) können.
- Erste einfache Abfragen zur Prüfung von Inhalten in Datenbanken mittels Data Query Language (DQL) kennenlernen.

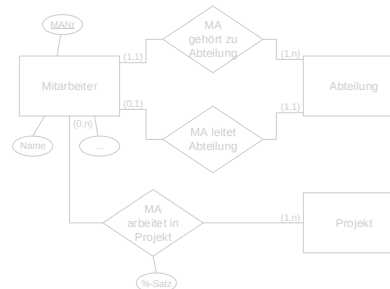
Themenüberblick



A Datenbankgrundlagen



B Konzeptionelle Modellierung



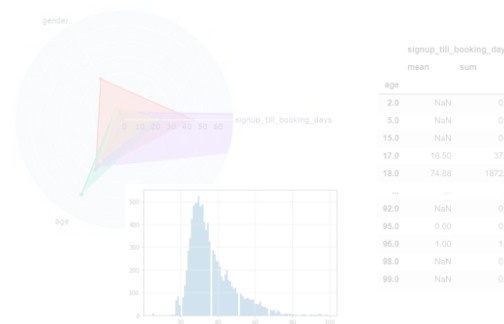
C Logische Modellierung & Normalisierung

MID	Name	Adresse	Gehalt
1234	Müller	Hofstraße 7	23.000 €
1235	Meyer	Hauptstraße 13b	45.678 €
1236	Schmidt	Kurze Straße 1	55.555 €
1237	Heinze	Neue Straße 34	44.333 €
1238	Mustermann	Musterstraße 8	80.000 €
1239	...	...	...

D Structured Query Language (SQL)

- filter your columns
SELECT col1, col2, col3, ... **FROM** table1
- filter the rows
WHERE col4 = 1 **AND** col5 = 2
- aggregate the data
GROUP by ...
- limit aggregated data
HAVING count(*) > 1
- order of the results
ORDER BY col2

E Analytics mit SQL



F Weitere Formen und Grundlagen der Datenhaltung



Agenda

■ Grundlagen von RDBMS und Bestandteile der SQL

☐ DDL: Anlegen von Datenbanken und Tabellen

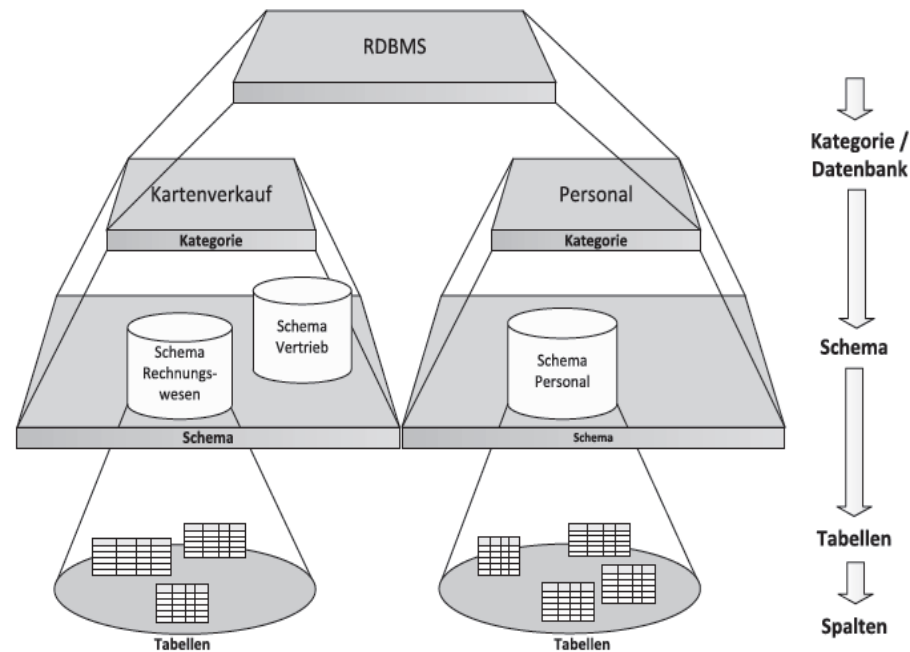
☐ DDL: Löschen von Tabellen

☐ DDL: Ändern von Tabellendefinitionen

☐ Zusammenfassung und Ausblick

Relationales DBMS

- Struktur relationaler Datenbankmanagementsysteme (RDBMS)



Structured Query Language

- SQL = Structured Query Language
- Deklarative Programmiersprache
 - Zweck: Datenmanagement und -abfrage in (insb. relationalen) Datenbanken
 - Algorithmische Lösung eines Problems steht nicht im Vordergrund
 - „Beschreibung“ der Aufgabenstellung
- Beispiel:
 - `SELECT Name FROM Kunde;`
- Standardsprache für das relationale Datenbankmodell
 - Aber: es gibt einige „Dialekte“

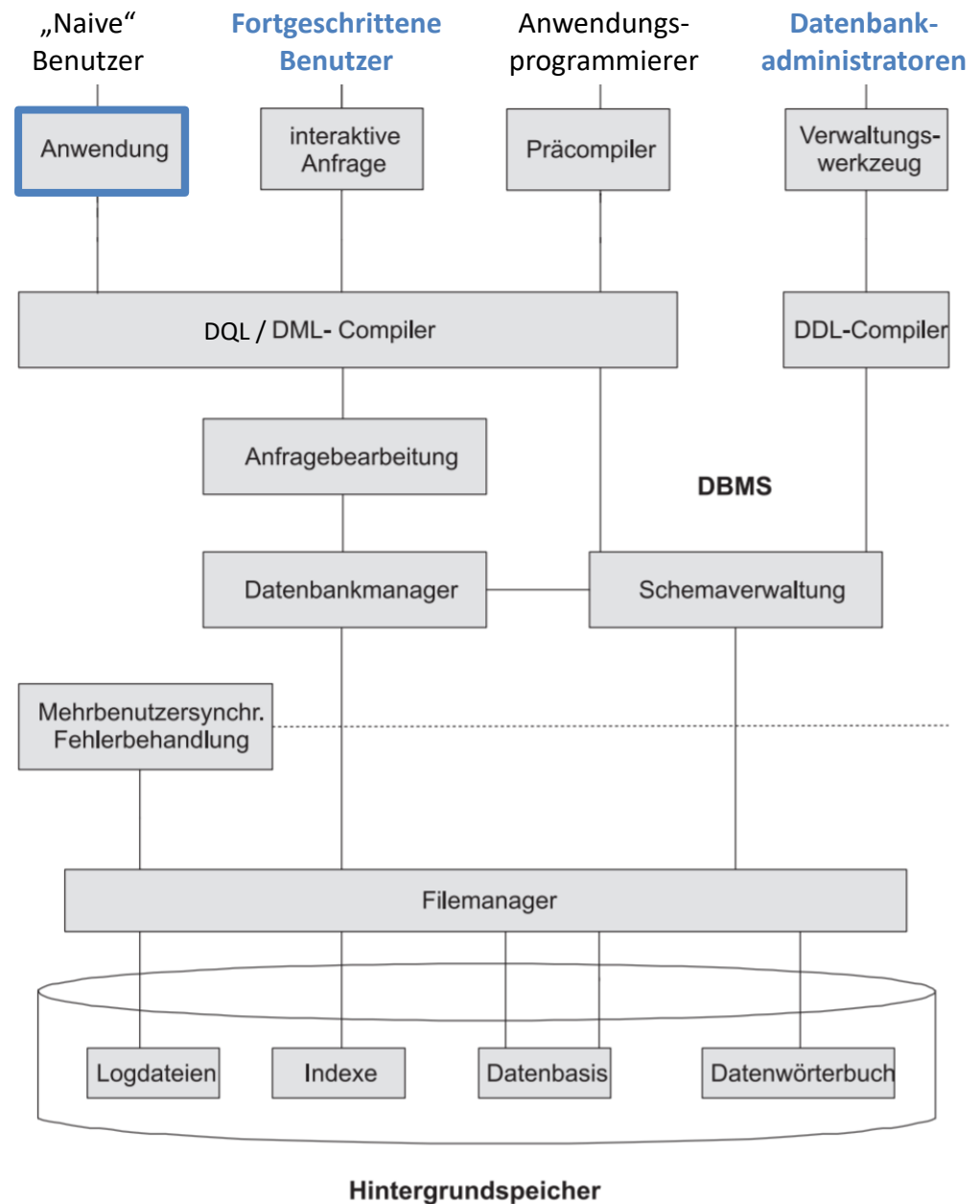
Relationales DBMS

- Teilbereiche und beispielhafte Befehle der SQL zur Steuerung und Abfrage von RDBMS

DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

Relationales DBMS

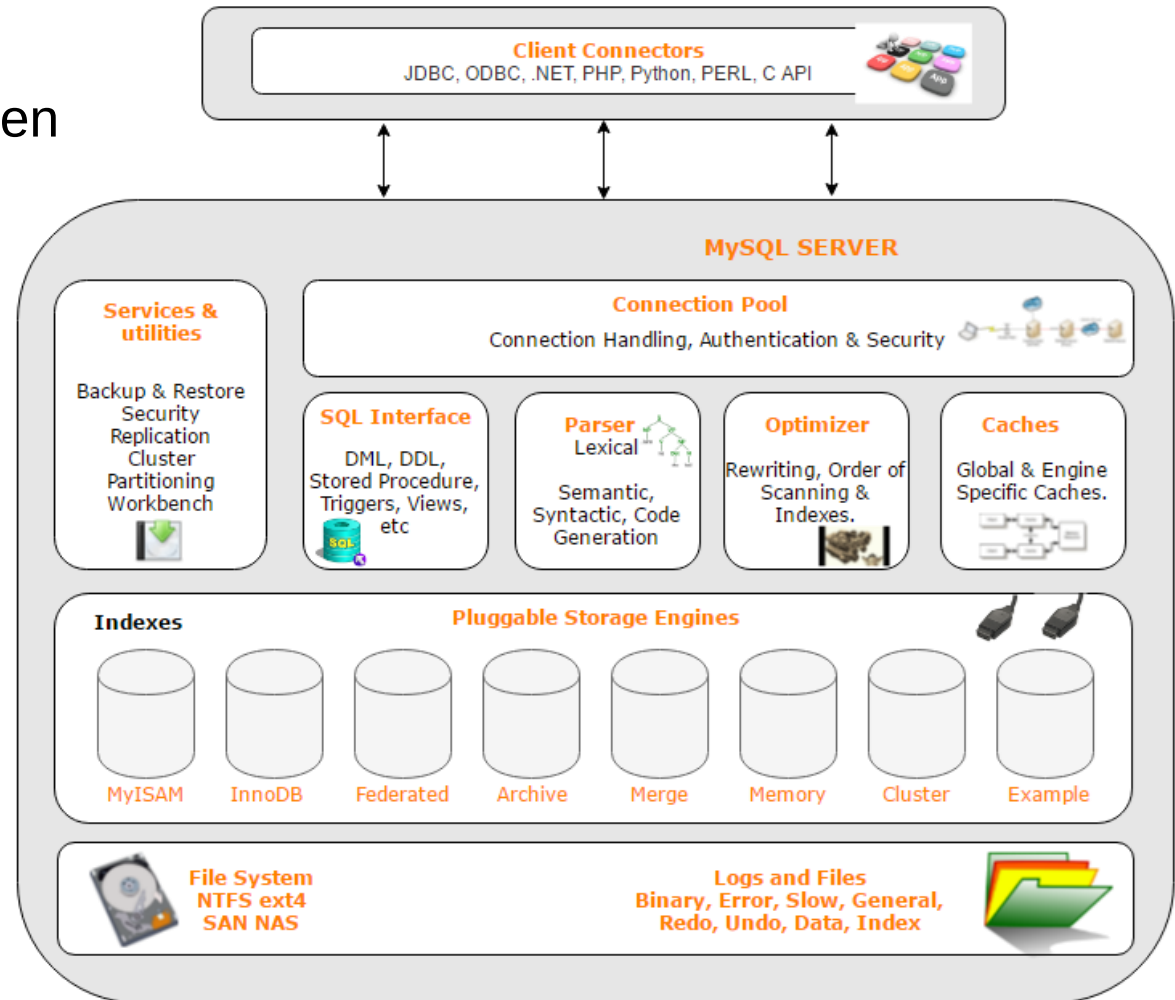
- Typische Komponenten von RDBMS



Quelle: Vornberger 2015

Relationales DBMS

- Typische Komponenten von RDBMS
 - Beispiel MySQL



Quelle: <https://www.rathishkumar.in/2016/04/understanding-mysql-architecture.html>

Agenda

- ☐ Grundlagen von RDBMS und Bestandteile der SQL
- ☒ DDL: Anlegen von Datenbanken und Tabellen
- ☐ DDL: Löschen von Tabellen
- ☐ DDL: Ändern von Tabellendefinitionen
- ☐ Zusammenfassung und Ausblick

Übung “Cloud SQL Setup”



- Übung (<https://pkuep.github.io/hska-140-db>)
 - Nun möchten wir die theoretischen Konzepte in einer „echten“ Umgebung umsetzen.

DSCB140 - Datenbanken und Datenkunde

Einführung

[Herausforderungen dateibasierter Speicherung und erster Datenbankkontakt](#)

Normalisierung

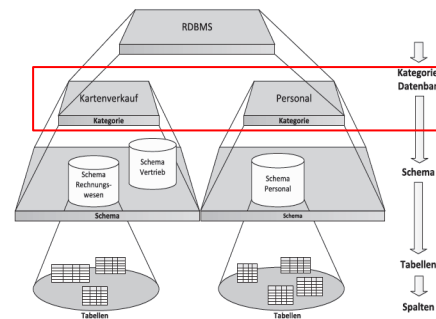
[Übung Normalisierung](#)

SQL



[Setup der Übungsumgebung](#)

Relationales DBMS



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen einer Datenbank

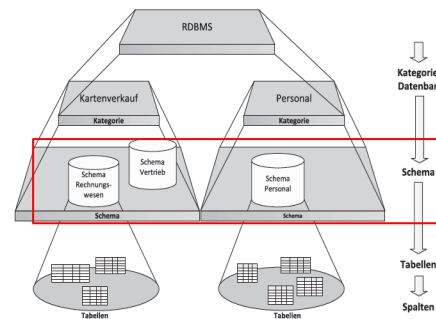
- Die genaue Syntax ist abhängig vom verwendeten RDBMS.
 - Wir verwenden die Syntax von MySQL als Einstieg.
 - „Syntax Conventions“: <https://dev.mysql.com/doc/refman/5.7/en/manual-info.html>

```
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] db_name
```

- Hier zeigt sich bereits eine Besonderheit von MySQL: Datenbank und Schema werden synonym verwendet.
 - Wir können entweder CREATE DATABASE db_name oder CREATE SCHEMA db_name verwenden. Beides führt dazu, dass eine Datenbank erzeugt wird.
 - Es gibt bei MySQL nicht die oben dargestellte Schicht „Schema“.
 - Andere RDBMS bieten diese Funktionalität, so dass CREATE SCHEMA etwas anderes bewirkt als CREATE DATABASE.
- Wenn der db_name bereits vergeben ist schlägt der Befehl fehl.
 - Mit „IF NOT EXISTS“ steuern wir, dass die Datenbank nur angelegt wird, wenn sie nicht schon existiert. Dies gibt es auch für Tabellen (s.u.).
- Mit „use db_name“ wechseln wir zu der Datenbank (siehe Übung).

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

Relationales DBMS

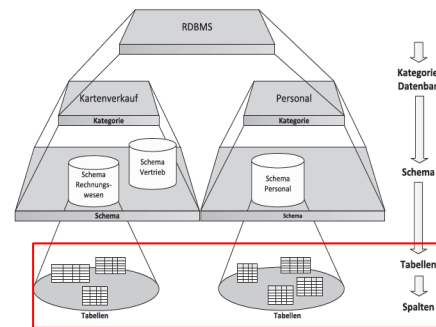


DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen eines Schemas

- Die genaue Syntax ist abhängig vom verwendeten RDBMS.
 - Bei MySQL existiert diese Schicht nicht.
 - In PostgreSQL kann ein Schema wie folgt angelegt werden:
 - `CREATE SCHEMA schema_name`
 - “A schema is essentially a namespace: it contains named objects (tables, data types, functions, and operators) whose names can duplicate those of other objects existing in other schemas.” (<https://www.postgresql.org/docs/current/sql-createschema.html>)

Relationales DBMS



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen von Tabellen

- Die genaue Syntax ist abhängig vom verwendeten RDBMS.
 - Strukturell sind die DDL-Queries ähnlich, aber insbesondere im Hinblick auf die Datentypen sowie die Syntax zum Festlegen von Schlüsseln und Constraints unterscheidet sich teilweise stark.
 - Wir fokussieren zunächst auf die MySQL-Syntax und darin auch nur auf den Basis-Befehl.

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name  
(create_definition,...)
```

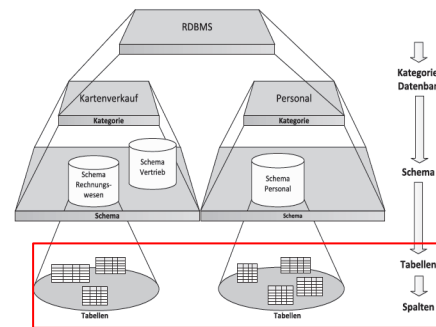
Beispiel:

```
CREATE TABLE rechnung (  
...  
)
```

- Wir können also auch temporäre Tabellen anlegen. Auch können wir definieren, dass Tabellen nur angelegt werden, wenn sie nicht existieren.
- Die „create_definition“ spezifiziert die anzulegenden Spalten und deren Datentypen (also das Relationenschema).
 - Je Spalte ist eine create_definition zu spezifizieren.
 - Siehe nächste Folie
 - Mehrere Spaltendefinitionen werden durch Komma getrennt.

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

Relationales DBMS



DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen von **Spalten** (Teil 1)

- Die `create_definition` lässt komplexe Definitionen zu.
 - Für uns ist zunächst insbesondere `col_name column_definition` (s.u.) interessant.
 - Darüber hinaus ist für uns wichtig wie Primärschlüssel (PRIMARY KEY) und Fremdschlüssel (FOREIGN KEY) definiert werden können:

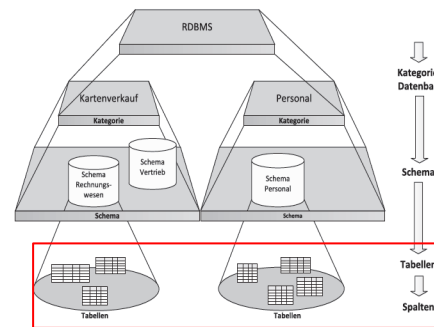
```
create_definition: {
    col_name column_definition
| {INDEX | KEY} [index_name] [index_type] (key_part,...)
    [index_option] ...
| {FULLTEXT | SPATIAL} [INDEX | KEY] [index_name] (key_part,...)
    [index_option] ...
| [CONSTRAINT [symbol]] PRIMARY KEY
    [index_type] (key_part,...)
    [index_option] ...
| [CONSTRAINT [symbol]] UNIQUE [INDEX | KEY]
    [index_name] [index_type] (key_part,...)
    [index_option] ...
| [CONSTRAINT [symbol]] FOREIGN KEY
    [index_name] (col_name,...)
    reference_definition
| CHECK (expr)
}
```

Beispiel:

```
CREATE TABLE rechnung (
    rid ...,
    redatum ...,
    mitarbeiter ...,
    primary key (rid),
    foreign key (mitarbeiter) ...
)
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

Relationales DBMS



DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen von **Spalten** (Teil 2)

- Die column_definition enthält insbesondere die Definition des Datentyps und ob die Spalte „NOT NULL“ gesetzt werden soll (also nicht leer sein darf).
 - Darüber hinaus können Standardwerte (default_value) gesetzt und das automatische Hochzählen (auto_increment) aktiviert werden.
 - Auch das Setzen einer Primärschlüsselspalte kann hier vereinfacht vorgenommen werden.

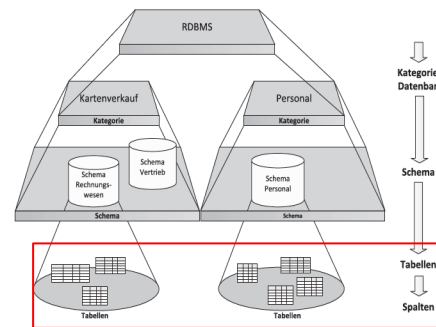
```
column_definition: {
    data_type [NOT NULL | NULL] [DEFAULT default_value]
    [AUTO_INCREMENT] [UNIQUE [KEY]] [[PRIMARY] KEY]
    [COMMENT 'string']
    [COLLATE collation_name]
    [COLUMN_FORMAT {FIXED | DYNAMIC | DEFAULT}]
    [STORAGE {DISK | MEMORY}]
    [reference_definition]
    | data_type
    [COLLATE collation_name]
    [GENERATED ALWAYS] AS (expr)
    [VIRTUAL | STORED] [NOT NULL | NULL]
    [UNIQUE [KEY]] [[PRIMARY] KEY]
    [COMMENT 'string']
    [reference_definition]
}
```

Beispiel:

```
CREATE TABLE rechnung (
    rid ... auto_increment,
    redatum ...,
    mitarbeiter ... not null,
    primary key (rid),
    foreign key (mitarbeiter) ...
)
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

Relationales DBMS



DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

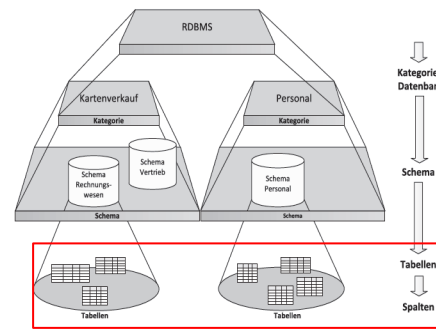
■ Anlegen von **Spalten** (Teil 3)

- SQL definiert einige „Standarddatentypen“
→ Achtung: nicht alle RDBMS setzen diese um und haben teilweise eigene Bezeichnungen!

Data type	Description
CHARACTER(n)	Character string. Fixed-length n
VARCHAR(n) or CHARACTER VARYING(n)	Character string. Variable length. Maximum length n
BINARY(n)	Binary string. Fixed-length n
BOOLEAN	Stores TRUE or FALSE values
VARBINARY(n) or BINARY VARYING(n)	Binary string. Variable length. Maximum length n
INTEGER(p)	Integer numerical (no decimal). Precision p
SMALLINT	Integer numerical (no decimal). Precision 5
INTEGER	Integer numerical (no decimal). Precision 10
BIGINT	Integer numerical (no decimal). Precision 19

DECIMAL(p,s)	Exact numerical, precision p, scale s. Example: decimal(5,2) is a number that has 3 digits before the decimal and 2 digits after the decimal
NUMERIC(p,s)	Exact numerical, precision p, scale s. (Same as DECIMAL)
FLOAT(p)	Approximate numerical, mantissa precision p. A floating number in base 10 exponential notation. The size argument for this type consists of a single number specifying the minimum precision
REAL	Approximate numerical, mantissa precision 7
FLOAT	Approximate numerical, mantissa precision 16
DOUBLE PRECISION	Approximate numerical, mantissa precision 16
DATE	Stores year, month, and day values
TIME	Stores hour, minute, and second values
TIMESTAMP	Stores year, month, day, hour, minute, and second values
INTERVAL	Composed of a number of integer fields, representing a period of time, depending on the type of interval
ARRAY	A set-length and ordered collection of elements
MULTISET	A variable-length and unordered collection of elements
XML	Stores XML data

Relationales DBMS



DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

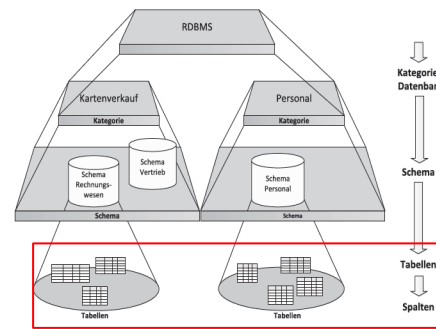
- Anlegen von **Spalten** (Teil 4)
 - MySQL bietet folgende **Datentypen zur Speicherung von Text**

Data type	Description
CHAR(size)	Holds a fixed length string (can contain letters, numbers, and special characters). The fixed size is specified in parenthesis. Can store up to 255 characters
VARCHAR(size)	Holds a variable length string (can contain letters, numbers, and special characters). The maximum size is specified in parenthesis. Can store up to 255 characters. Note: If you put a greater value than 255 it will be converted to a TEXT type
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT	Holds a string with a maximum length of 65,535 characters
BLOB	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LOBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(x,y,z,etc.)	Let you enter a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. Note: The values are sorted in the order you enter them. You enter the possible values in this format: ENUM('X','Y','Z')
SET	Similar to ENUM except that SET may contain up to 64 list items and can store more than one choice

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/data-types.html>

DDL: Anlegen von Datenbanken und Tabellen

Relationales DBMS



DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen von **Spalten** (Teil 5)

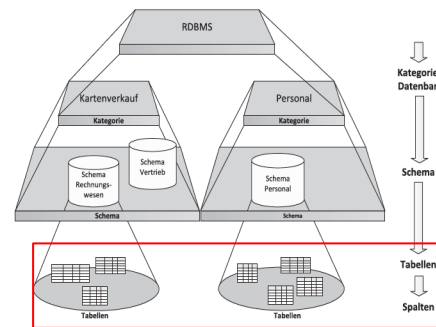
- MySQL bietet folgende **Datentypen zur Speicherung numerischer Werte**

Data type	Description
TINYINT(size)	-128 to 127 normal. 0 to 255 UNSIGNED*. The maximum number of digits may be specified in parenthesis
SMALLINT(size)	-32768 to 32767 normal. 0 to 65535 UNSIGNED*. The maximum number of digits may be specified in parenthesis
MEDIUMINT(size)	-8388608 to 8388607 normal. 0 to 16777215 UNSIGNED*. The maximum number of digits may be specified in parenthesis
INT(size)	-2147483648 to 2147483647 normal. 0 to 4294967295 UNSIGNED*. The maximum number of digits may be specified in parenthesis
BIGINT(size)	-9223372036854775808 to 9223372036854775807 normal. 0 to 18446744073709551615 UNSIGNED*. The maximum number of digits may be specified in parenthesis
FLOAT(size,d)	A small number with a floating decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter
DOUBLE(size,d)	A large number with a floating decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter
DECIMAL(size,d)	A DOUBLE stored as a string , allowing for a fixed decimal point. The maximum number of digits may be specified in the size parameter. The maximum number of digits to the right of the decimal point is specified in the d parameter

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/data-types.html>

DDL: Anlegen von Datenbanken und Tabellen

Relationales DBMS



DDL Data Definition Language	DQL Data Query Language	DML Data Manipulation Language	DCL Data Control Language	TCL Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen von **Spalten** (Teil 6)

- MySQL bietet folgende **Datentypen zur Speicherung von Datums- und Zeitwerten**

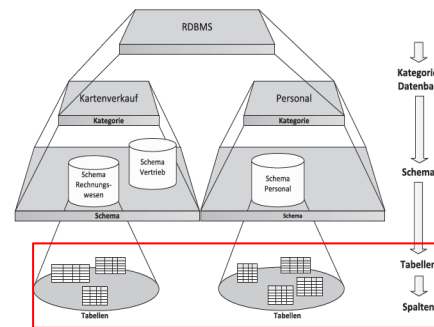
Data type	Description
DATE()	A date. Format: YYYY-MM-DD Note: The supported range is from '1000-01-01' to '9999-12-31'
DATETIME()	*A date and time combination. Format: YYYY-MM-DD HH:MI:SS Note: The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'
TIMESTAMP()	*A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD HH:MI:SS Note: The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC
TIME()	A time. Format: HH:MI:SS Note: The supported range is from '-838:59:59' to '838:59:59'
YEAR()	A year in two-digit or four-digit format. Note: Values allowed in four-digit format: 1901 to 2155. Values allowed in two-digit format: 70 to 69, representing years from 1970 to 2069

Beispiel:

```
CREATE TABLE rechnung (
  rid int auto_increment,
  redatum date,
  mitarbeiter int not null,
  primary key (rid),
  foreign key (mitarbeiter) ...
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/data-types.html>

Relationales DBMS



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

■ Anlegen von **Spalten** (Teil 7)

- Achtung: es gibt weitere spezialisierte Datentypen, die MySQL aufnehmen kann
 - Beispiele: geospatial Data, JSON (dazu später mehr)
- Zur Definition von Fremdschlüsseln benötigen wir die `reference_definition`.

```
reference_definition:
REFERENCES tbl_name (key_part,...)
[MATCH FULL | MATCH PARTIAL | MATCH SIMPLE]
[ON DELETE reference_option]
[ON UPDATE reference_option]
```

- Diese erlaubt uns zu definieren, welche Attribute der anzulegenden Tabelle als Fremdschlüssel auf welche Primärschlüsselattribute anderer Relationen zeigen.

Beispiel:

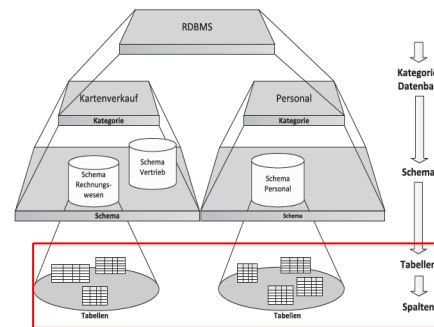
```
CREATE TABLE rechnung (
  rid int auto_increment,
  redatum date,
  mitarbeiter int not null,
  primary key (rid),
  foreign key (mitarbeiter) references mitarbeiter (mid)
)
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

Agenda

- ☐ Grundlagen von RDBMS und Bestandteile der SQL
- ☐ DDL: Anlegen von Datenbanken und Tabellen
- ☒ DDL: Löschen von Tabellen
- ☐ DDL: Ändern von Tabellendefinitionen
- ☐ Zusammenfassung und Ausblick

Relationales DBMS



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

- Zum Löschen von Tabellen (und auch Datenbanken) wird der DROP-Befehl verwendet:

```
DROP [TEMPORARY] TABLE [IF EXISTS]
tbl_name [, tbl_name] ...
```

- Um Fehlermeldungen zu vermeiden kann „if exists“ verwendet werden.
 - Nützlich, wenn mehrere Tabellen gelöscht werden und manche evtl. nicht existieren.
- Datenbanken werden so gelöscht:

```
DROP {DATABASE | SCHEMA} [IF EXISTS] db_name
```

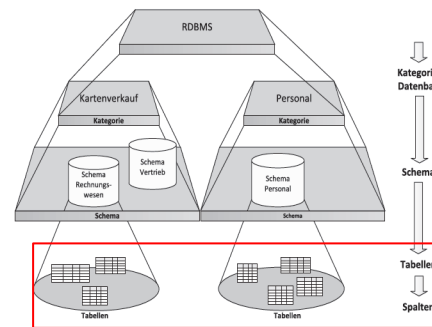
 - Merke: Database = Schema in MySQL!

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/drop-table.html>

Agenda

- ☐ Grundlagen von RDBMS und Bestandteile der SQL
- ☐ DDL: Anlegen von Datenbanken und Tabellen
- ☐ DDL: Löschen von Tabellen
- ☒ DDL: Ändern von Tabellendefinitionen
- ☐ Zusammenfassung und Ausblick

Relationales DBMS



DDL	DQL	DML	DCL	TCL
Data Definition Language	Data Query Language	Data Manipulation Language	Data Control Language	Transaction Control Language
CREATE	SELECT	INSERT	GRANT	COMMIT
ALTER		UPDATE	REVOKE	ROLLBACK
DROP		DELETE		SAVEPOINT
RENAME		MERGE		SET TRANSACTION
TRUNCATE		CALL		
COMMENT		EXPLAIN PLAN		
		LOCK TABLE		

- Mit dem ALTER-Statement können Tabellendefinitionen angepasst werden.
 - Vorteil: Anpassungen im laufenden Betrieb möglich.
 - ALTER TABLE ist ähnlich komplex wie CREATE:

■ Beispiele

- Hinzufügen einer Spalte und gleichzeitig eines Fremdschlüssels zur Tabelle Rechnung

Beispiel:

```
ALTER TABLE rechnung
ADD COLUMN kunde_id integer NOT NULL,
ADD FOREIGN KEY (kunde_id) references kunde(kid)
```

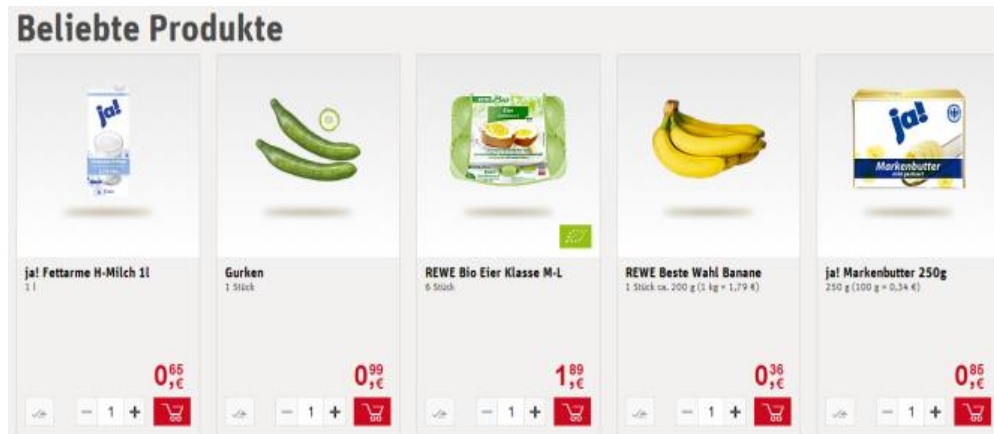
```
ALTER TABLE tbl_name
[alter_option [, alter_option] ...]
[partition_options]

alter_option: {
    table_options
    | ADD [COLUMN] col_name column_definition
      [FIRST | AFTER col_name]
    | ADD [COLUMN] (col_name column_definition, ...)
    | ADD {INDEX | KEY} [index_name]
      [index_type] (key_part, ...) [index_option] ...
    | ADD {FULLTEXT | SPATIAL} [INDEX | KEY] [index_name]
      (key_part, ...) [index_option] ...
    | ADD [CONSTRAINT [symbol]] PRIMARY KEY
      [index_type] (key_part, ...)
      [index_option] ...
    | ADD [CONSTRAINT [symbol]] UNIQUE [INDEX | KEY]
      [index_name] [index_type] (key_part, ...)
      [index_option] ...
    | ADD [CONSTRAINT [symbol]] FOREIGN KEY
      [index_name] (col_name, ...)
      reference_definition
    | ADD CHECK (expr)
    | ALGORITHM [=] {DEFAULT | INPLACE | COPY}
    | ALTER [COLUMN] col_name {SET DEFAULT literal | DROP DEFAULT}
    | CHANGE [COLUMN] old_col_name new_col_name column_definition
      [FIRST | AFTER col_name]
    | [DEFAULT] CHARACTER SET [=] charset_name [COLLATE [=] collation,
    | CONVERT TO CHARACTER SET charset_name [COLLATE collation_name]
    | {DISABLE | ENABLE} KEYS
    | {DISCARD | IMPORT} TABLESPACE
    | DROP [COLUMN] col_name
    | DROP {INDEX | KEY} index_name
    | DROP PRIMARY KEY
    | DROP FOREIGN KEY fk_symbol
    | FORCE
    | LOCK [=] {DEFAULT | NONE | SHARED | EXCLUSIVE}
    | MODIFY [COLUMN] col_name column_definition
      [FIRST | AFTER col_name]
    | ORDER BY col_name [, col_name] ...
    | RENAME {INDEX | KEY} old_index_name TO new_index_name
    | RENAME [TO | AS] new_tbl_name
    | {WITHOUT | WITH} VALIDATION
}
```

Quelle: <https://dev.mysql.com/doc/refman/5.7/en/alter-table.html>

Fallstudie

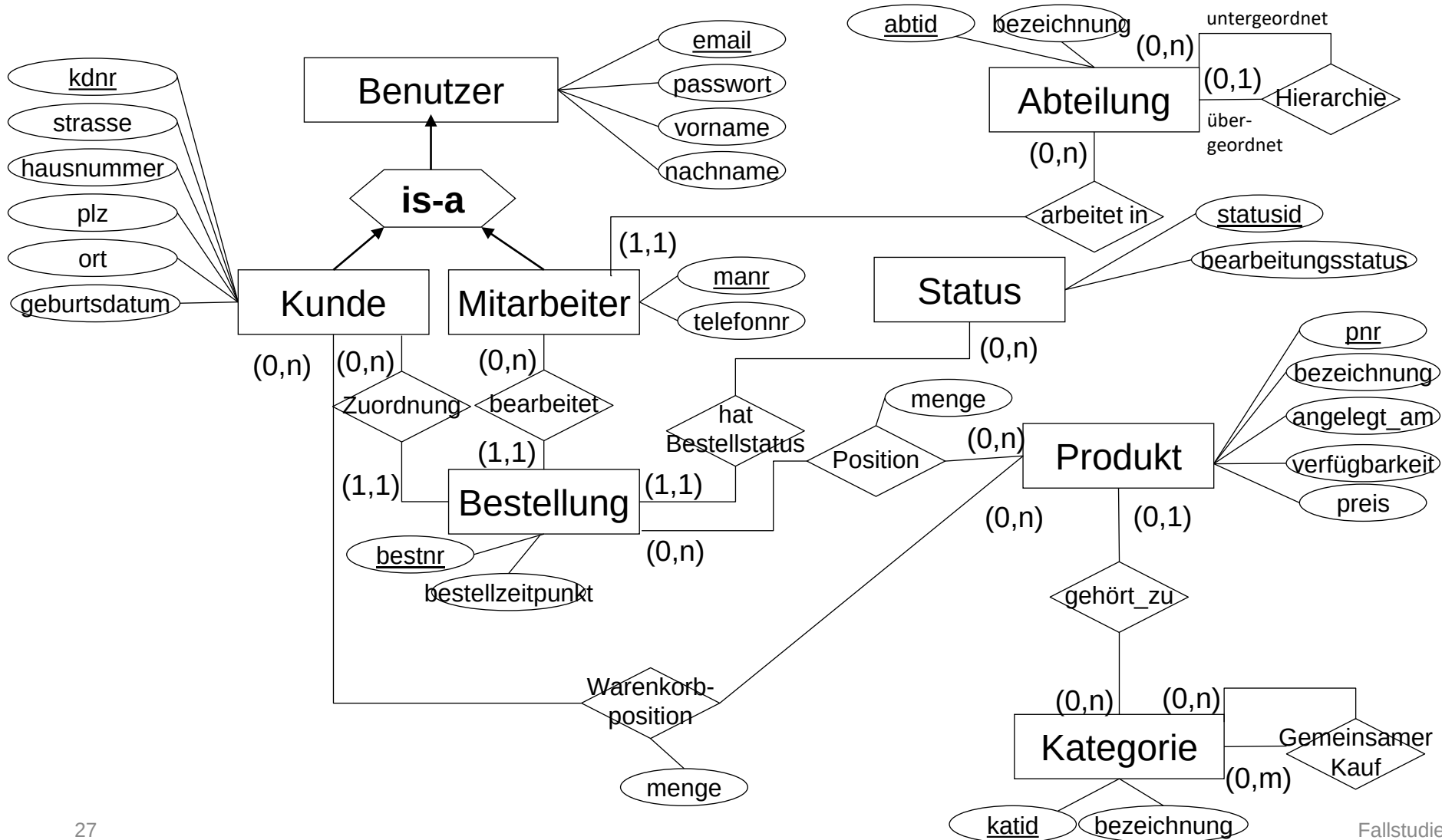
- Wir möchten nun sukzessive das Datenmodell für unseren Webshop aufbauen.



- Wir verwenden das folgende ERM bzw. die entsprechende Definition des relationalen Modells.

Bitte nutzen Sie
dieses ERM

Gesamtes
Modell in
(min, max)-
Notation



Bitte nutzen Sie dieses relationale Modell

Folgende Informationen stehen Ihnen bezüglich Datentypen und weiterer Nebenbedingungen zur Verfügung:

Benutzer: (email VARCHAR(100), passwort VARCHAR(50) (**NOT NULL**), vorname VARCHAR(50) (**NOT NULL**), nachname VARCHAR(50) (**NOT NULL**))

Mitarbeiter: (#email ???, mannr INT (**UNIQUE**, **NOT NULL**), telefonnr VARCHAR(30), #abtid INT (**NOT NULL**))

Kunde: (#email ???, kdnr INT (**UNIQUE**, **NOT NULL**), strasse VARCHAR(50), hausnummer VARCHAR(10), plz VARCHAR(5), ort VARCHAR(50), geburtsdatum DATE)

Abteilung: (abtid INT, bezeichnung VARCHAR(50), uebergeordnet#abtid ???)

Status: (statusid INT, bearbeitungsstatus VARCHAR(50))

Kategorie: (katid INT, bezeichnung VARCHAR(50))

GemeinsamerKauf: (k1#katid ???, k2#katid ???)

Produkt: (pnr INT, bezeichnung VARCHAR(50), angelegt_am TIMESTAMP, verfuegbarkeit INT (**NOT NULL**), preis DECIMAL(6,2) (**NOT NULL**), #katid ???)

WarenkorbPosition: (#email ???, #pnr ???, menge INT (**NOT NULL**))

Bestellung: (bestnr INT (**AUTO_INCREMENT**), bestellzeitpunkt TIMESTAMP, #kunde ??? (**NOT NULL**), bearbeitet#email ??? (**NOT NULL**), #statusid ??? (**NOT NULL**))

BestellPosition: (#bestnr ???, #pnr ???, menge INT (**NOT NULL**))

Weitere Anforderungen

- ??? bedeutet, dass Sie den Datentyp selbst herausfinden müssen.
- UNIQUE bedeutet, dass der Attributwert eindeutig sein muss (siehe VL3)
- NOT NULL bedeutet, dass der Attributwert gesetzt werden muss (siehe VL3)
- AUTO_INCREMENT bedeutet, dass der Wert der Spalte automatisch hochgezählt werden soll.

Aufgabe: setzen Sie das relationale Modell des Webshops in Ihrer Datenbank um.

Schauen Sie sich <https://dev.mysql.com/doc/refman/5.7/en/create-table.html> an und versuchen Sie, die besonderen Anforderungen umzusetzen.

Übung “SQL DDL”



- Übung (<https://pkuep.github.io/hska-140-db>)
 - Wenn Sie Hilfestellung benötigen bzw. zum Lösungsvergleich, können Sie sich folgendes Codelab anschauen.

DSCB140 - Datenbanken und Datenkunde

Einführung

[Herausforderungen dateibasierter Speicherung und erster Datenbankkontakt](#)

Normalisierung

[Übung Normalisierung](#)

SQL

Allgemein

[Setup der Übungsumgebung](#)

Data Definition Language

 [Fallstudie Webshop - Tabellenerzeugung](#)

Agenda

- ☐ Grundlagen von RDBMS und Bestandteile der SQL
- ☐ DDL: Anlegen von Datenbanken und Tabellen
- ☐ DDL: Löschen von Tabellen
- ☐ DDL: Ändern von Tabellendefinitionen
- ☒ Zusammenfassung und Ausblick



- In der letzten Vorlesung haben wir uns intensiv mit der Data Definition Language anhand von MySQL auseinandergesetzt und Sie haben ein Relationenschema in ein konkretes DBMS überführt.
- Zusätzlich haben wir unsere Cloud-Umgebung für SQL eingerichtet.
- Sie sollten nun
 - Tabellen in MySQL anlegen können,
 - die existierenden Datentypen kennen und wissen, wann sie anzuwenden sind,
 - spezielle Schlüsselwörter z.B. zur Definition von Schlüsseln, eindeutiger Werte, sowie weiterer Constraints kennen,
 - die unterschiedlichen Optionen der referentiellen Integrität kennen und einstellen können, sowie
 - existierende Tabellen hinsichtlich ihrer Definition anpassen können.
- In der folgenden Veranstaltung werden wir unsere Datenbank „mit Leben füllen“. Dazu nutzen wir die Data Manipulation Language von SQL.

Vielen Dank für Ihre Aufmerksamkeit!

